

숫자 손실 설계 기록

손실을 세 번 고쳤고, 그 사이에 멀쩡한 모델을 망가졌다고 한 번 오진했습니다
무엇을 재서 그렇게 보였고, 그 측정이 왜 거짓이었는가

이 프로젝트의 정답은 숫자입니다. 거리, 방위각, 박스 좌표, 속도. 측정하면 정답 토큰의 44~88%가 숫자와 구분자입니다. 그런데 크로스엔트로피는 638 을 요구하면서 639 와 100 을 거의 똑같이 벌줍니다 — argmax 가 정답인지만 묻기 때문입니다. 그것을 고치려는 세 번의 시도를 기록합니다.

0. 세 번의 시도

	무엇	결과
① 자릿수 거리	digit_distance_loss — 0~9 열 개 토큰에만 걸고 자릿수 값의 거리를 벌점	가중치 0 으로 두고 쓰지 않음. 자릿수마다 따로 채점하므로 100 대신 900 과 200 이 같은 벌점
② 기댓값 거리	숫자 하나를 토큰 하나로 만들고 $E[v]$ 정답! 을 벌점	식이 틀렸음. argmax 가 어긋난 분포에 만점을 줌 — 구성한 분포로 직접 확인. 다만 이것이 학습을 망쳤다는 증거는 없음 (4 절)
③ 거리 기댓값	$E[v - \text{정답}]$ — 정답에 확률을 몰아야만 0	속임수 분포가 0.0000 $\rightarrow$ 1.0000. 검증 완료, 재학습은 이후

1. 왜 숫자를 따로 다루는가

정답에서 숫자가 차지하는 비중을 먼저 짚습니다.

태스크	정답 토큰	숫자·구분자	비율
plan_ego_xy	41	36	87.8%
motion_seg_bbox	147	115	78.2%
det_objects_3dbbox	262	203	77.4%
track_step_bbox	154	116	75.3%
det_objects_azdeg	75	33	43.9%

Qwen3-VL 은 0~9 만 단일 토큰이라 숫자가 자릿수로 쪼개집니다.

'-13.0' → '-' | '1' | '3' | '.' | '0' 5 토큰
'[301, 495, 371, 569]' → 24 토큰 중 20 개가 숫자와 구분자

두 가지가 동시에 걸립니다. **토큰이 낭비되고**, 자릿수마다 독립적인 151,936 지선다가 되어 **값의 가까움이 손실에 나타나지 않습니다**.

2. 숫자 토큰 2,503개

데이터에 실제로 나오는 숫자를 세어 99.9% 를 덮는 2,503개(소수 1,321 + 정수 1,182)를 어휘에 넣었습니다. 어휘 151,669 → 154,165.

문자열	이전	이후
automobile 24 m az +13 deg	11	9
automobile (9.7, -13.0, 0.9) size 5.3x2.2x1.9 m yaw -179 deg	38	23
#12 automobile [301, 495, 371, 569]	24	15
+1s (+8.8, +0.0)	13	9

2.1 값 기반 초기화 — 첫 시도는 틀렸습니다

토큰을 더하는 것만으로는 "값이 가까우면 비슷하다"가 생기지 않습니다. 먼저 **자릿수 임베딩의 평균**으로 초기화해 재봤습니다.

	자릿수 평균	값 기반 (sin/cos)
100 ↔ 110	0.916	+0.106
100 ↔ 200	0.933 ← 더 먼데 더 비슷	+0.024
100 ↔ 900	0.889	0.008
값 차이 · 유사도 상관	0.292	0.706

자릿수 평균은 '1', '0', '0' 과 '2', '0', '0' 이 두 자리를 공유하는 것만 반영하고 크기는 반영하지 않습니다. 값을 여러 주파수의 sin/cos 로 펼치면 값 차이가 커질수록 유사도가 단조롭게 떨어집니다.

3. 두 번째 시도가 틀린 지점

숫자가 한 토큰이 됐으니 손실도 값 위에서 매길 수 있습니다. 그렇게 쓴 것이 이것입니다.

$loss = \sum p(v) \cdot v \quad \text{정답} \mid \# \text{ 기댓값의 거리}$

이 식은 기댓값만 맞으면 0 입니다. 정답이 100 일 때 0 에 50%, 200 에 50% 를 주면 기댓값이 100 이라 손실이 0 인데, 생성이 뽑는 argmax 는 0 이나 200 입니다.

분포 (정답 = 100)	argmax	옛 손실	새 손실
100 에 전부	100	0.0000	0.0000
0 과 200 에 반반	0	0.0000	1.0000
0·200 반반 + 100 조금	0	0.0000	0.9000
50 과 200	50	0.0000	0.6665
전부 균등	0	1.0000	2.1000
900 에 전부	900	7.5000	8.0000

3.1 지표가 좋아지는 것을 성공으로 읽었습니다

학습 중 `train/number_distance` 가 1.133 → 0.004 로 내려갔고, 저는 그것을 "숫자 정확도가 100 배 좋아졌다"고 보고했습니다. 그 지표는 분포의 기댓값을 재고 생성은 argmax 를 씁니다. 둘이 갈라져 있는데 한쪽만 보고 있었습니다.

추적 문서에 "손실은 내려가는데 숫자 거리가 평평하면 형식만 배우는 중"이라고 적어 두었는데, 실제로 일어난 것은 그 반대였습니다 — 숫자 거리가 내려가면서 생성이 무너지는 경우. 그 조합은 예상하지 못했습니다.

4. 하루를 버린 오진

8,100 스텝 모델을 테스트 200 건씩 생성시켜 보고 **모델이 무너졌다고** 결론지었습니다. 나온 것이 이랬습니다.

```
plan_ego_xy
생성 +1s (+1, 0); +2s (+2, 0); +3s (+2, 0)
정답 +1s (+20.1, -0.0); +2s (+40.3, -0.0); +3s (+60.4, +0.1)

det_objects_azdeg
생성 automobile 9 m az 9 deg; (같은 문장 8 회 반복)
정답 automobile 17 m az +24 deg; automobile 23 m az +15 deg; ...
```

고유 숫자를 세니 태스크마다 **열 개**뿐이었습니다. 정답은 152~1,183 개의 서로 다른 값을 씁니다. 숫자 토큰 2,503 개를 넣어 놓고 한 자리 수로 돌아간 것으로 보였고, 손실 설계 결함이 8,100 스텝을 태웠다고 보고했습니다.

4.1 실제로는 디코더가 버리고 있었습니다

train_vlm 은 숫자 토큰 2,503 개를 어휘에 등록합니다. eval_all_tasks 는 **등록하지 않았**습니다. 모델은 학습한 대로 그 토큰들(id 151,672 이상)을 정상적으로 냈고, 그것을 모르는 토큰나이저가 디코딩에서 전부 버렸습니다. 화면에 남은 것은 살아남은 자릿수뿐이었습니다.

같은 체크포인트 · 같은 클립	나온 것
모델이 실제로 쓴 것 (번들 평가기)	+1s 20.6 m/s, yaw -0.0 deg/s
디코딩되어 보인 것 (저장소 평가기)	+1s 1 m/s, yaw 0 deg/s
정답	+1s 20.2 m/s, yaw +0.2 deg/s

plan_ego_control 속도 MAE	값
저장소 평가기 · 수정 전	7.59 m/s
저장소 평가기 · 수정 후	0.67 m/s
번들 평가기 (처음부터 등록하고 있었음)	0.97 m/s

두 평가 경로가 **같은 모델에 8 배 다른 점수**를 주고 있었고, 저는 그중 하나만 보고 모델을 판단했습니다. 번들 쪽은 처음부터 맞았는데, 두 결과가 어긋난다는 사실 자체를 확인하지 않았했습니다.

4.2 무엇이 살아남았나

	상태
손실 식 $ E[v] - \text{정답} $ 이 틀렸다는 것	유효 . 이 평가기가 아니라 구성된 분포로 직접 확인했습니다 (3 절)
그 손실이 학습을 망쳤다는 것	철회 . 근거로 든 것이 전부 디코더가 버린 자릿수였습니다
train/number_distance 1.133 → 0.004 가 속임수였다는 것	미확정 . 실제로 값을 잘 맞게 된 것일 수도 있습니다
두 체크포인트의 점수	전부 다시 잴 습니다. 5 절 — 모든 태스크에서 스텝이 많은 쪽이 낮습니다. 붕괴는 없었습니다

5. 두 체크포인트, 같은 조건

같은 테스트 200 건, 같은 생성 상한, 같은 채점기로 잰 값입니다. 차이는 학습 스텝뿐입니다.

태스크	지표	step 2,400	step 8,100	변화
det_objects_3dbbox	F1	0.100	0.267	개선
det_objects_azdeg	F1	0.117	0.276	개선
track_step_bbox	F1	0.609	0.679	개선
track_step_azdeg	F1	0.566	0.698	개선
motion_seg_bbox	F1	0.128	0.258	개선
motion_seg_azdeg	F1	0.091	0.248	개선
plan_ego_xy	변위 MAE (m)	1.096	0.914	개선
plan_ego_control	속도 MAE (m/s)	0.671	0.571	개선
agent_traj_azdeg	거리 MAE (m)	4.519	2.269	개선
qa	정답률	0.400	0.500	개선
qa_cot	정답률	0.405	0.565	개선

두 모델 모두 `--digit-weight 0.3` 으로, 같은 데이터로 학습했습니다. 차이는 스텝 수뿐입니다. 이 표는 숫자 토큰을 등록하도록 고친 평가기로 다시 잰 값이고, 앞서 보고했던 수치는 전부 폐기했습니다.

6. 고친 것과 남은 것

이전 $\text{loss} = \lvert \sum p(v) \cdot v - \text{정답} \rvert$ # 기댓값의 거리
이후 $\text{loss} = \sum p(v) \cdot \lvert v - \text{정답} \rvert$ # 거리의 기댓값

엔션 부등식으로 **거리의 기댓값 $\geq$ 기댓값의 거리** 이고, 옛 식은 느슨한 하한이었습니다. 모델은 그 틈에서 살고 있었습니다. 새 식은 정답 하나에 확률을 몰아야만 0 이 되므로 퍼뜨리면 반드시 값을 치릅니다.

남은 질문	왜 아직 모르나
새 손실로 학습하면 실제로 나아지는가	속임수 분포를 막는 것은 확인했지만, 학습에서 무엇이 나오는지 돌려봐야 합니다
옛 손실이 실제로 해를 끼쳤는가	5 절의 두 열이 그 답에 가장 가깝습니다. 스텝이 늘어도 나빠지지 않았다면 식은 틀렸어도 학습은 견뎌낸다는 뜻입니다
가중치 0.3 이 적절한가	새 손실로 0.3 과 0.05 를 나란히 돌려야 합니다
숫자 토큰 자체는 남길 만한가	토큰을 절반으로 줄이는 이득은 실측됐고, 오진의 원인은 토큰이 아니라 평가기였으므로 버릴 이유가 없습니다

6.1 측정 습관에 대해

생성 텍스트를 봤는데도 틀렸다는 점이 이번의 교훈입니다. 점수만 보지 말자는 원칙은 지켰고 실제로 텍스트를 열어 봤는데, **그 텍스트 자체가 디코더를 거치며 손상돼 있었습니다**. 원본이 손상됐다면 원본을 보는 것도 소용이 없습니다.

값싸게 잡을 수 있었던 지점이 하나 있었습니다. 번들 평가기와 저장소 평가기가 같은 체크포인트에 **8 배 다른 점수**를 주고 있었고, 그 사실은 두 결과를 나란히 놓기만 해도 보였습니다. 경로가 둘이면 반드시 한 번은 대조해야 합니다 — 어느 쪽이 맞는지 몰라도, 어긋난다는 것만 알면 멈출 수 있습니다.

문서 생성: `python -m datatools.build_digit_loss_report`